Performance Characterization of Recursive Quicksort on Out-of-Order Architectures

USF Bellini College of Artificial Intelligence, Cybersecurity and Computing
EEL 6764 - Computer Architecture
Team: Srikanth Akkaru, Mandar Dhamale, Johnathan Gutierrez-diaz, Sri Satya Sudarshan Varma Indukuri

1. Executive Summary & Storyline

Recursive Quicksort produces a demanding microarchitectural footprint due to unpredictable branch outcomes and volatile stack activity. This project characterizes the bottlenecks of a recursive Quicksort benchmark on a simulated out-of-order x86 processor (gem5). By isolating the sorting Region-of-Interest (ROI) via `m5_dump_reset_stats`, we precisely measured the stalls originating from branch mispredictions and L2 cache thrashing, and mapped the PPA tradeoffs required to mitigate them.

2. Methodology

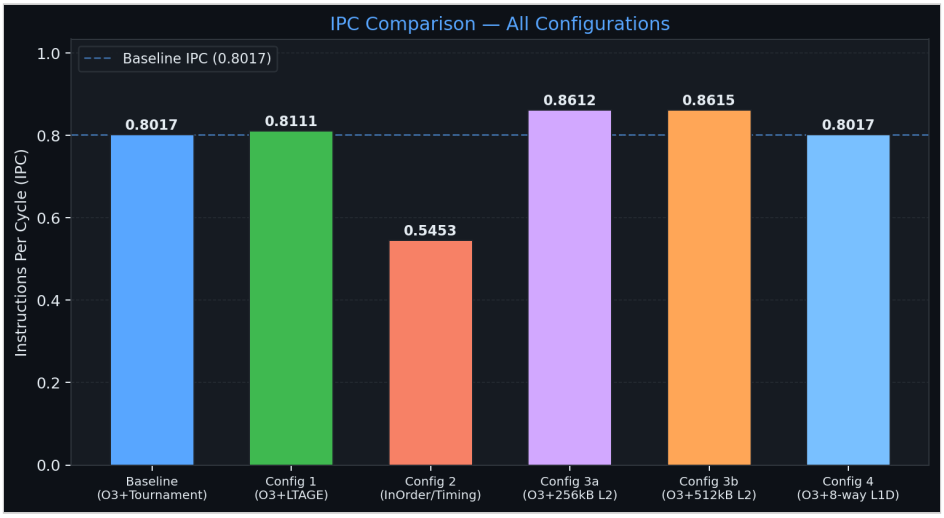
We simulated x86-64 execution in gem5 Syscall Emulation mode. The baseline CPU model was `03CPU` at 3 GHz, paired with an 8-way 16kB L1D cache and a 256kB L2. Simulation bounds were precisely managed with inline x86 assembly to reset statistics exactly before and after the `quickSort()` recursive tree.

3. Configuration Iterations

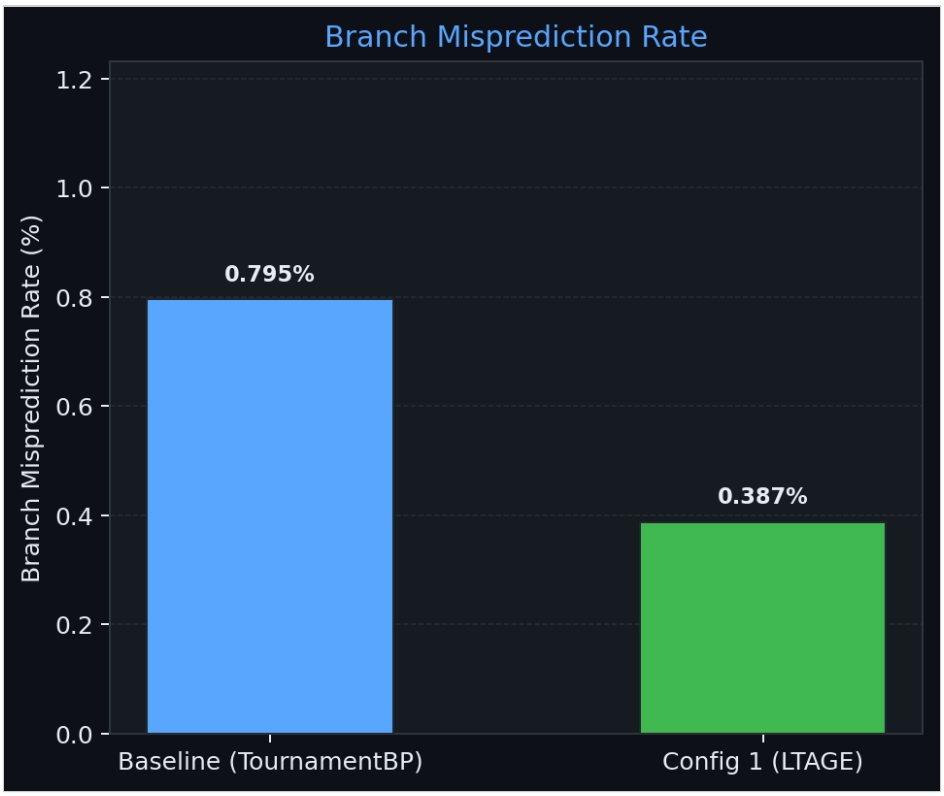
- **Config 1 (LTAGE Predictor):** Replaced TournamentBP to handle recursive branch patterns.
- **Config 2 (In-Order CPU):** Swapped O3 for TimingSimpleCPU to establish true out-of-order latency hiding benefits.
- **Config 3 (L2 Cache Scaling):** Enlarged array size to 100k, evaluating 256kB vs 512kB L2 sizes.
- **Config 4 (L1D Associativity):** Patched to explicitly test 8-way L1D cache geometries.

4. Performance Results

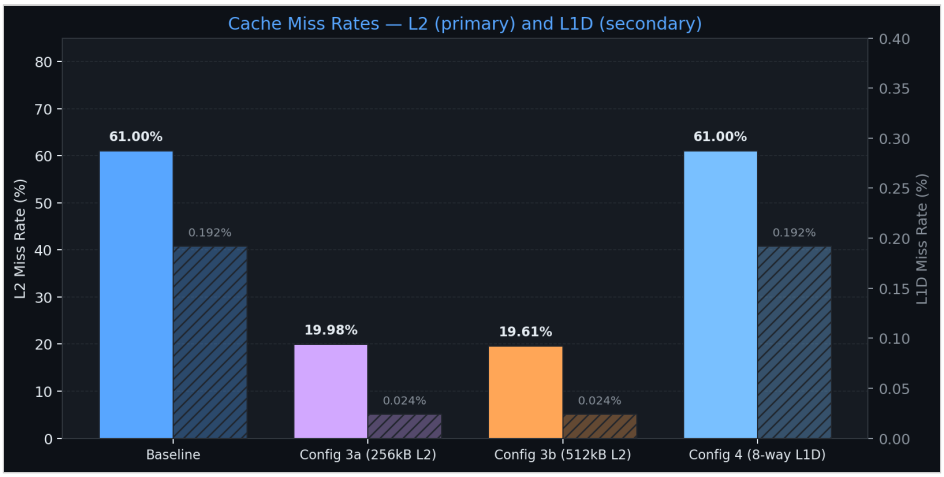
Configuration	n	IPC	Sim Ticks	Mispredict %	L2 Miss %
Baseline (O3, Tournament, 256k)	10k	0.8017	301M	0.795%	61.00%
Config 1 (O3, LTAGE, 256k)	10k	0.8111	298M	0.387%	61.04%
Config 2 (In-Order, 256k)	10k	0.5453	443M	N/A	58.11%
Config 3a (O3, Tourn, 256k)	100k	0.8612	2,747M	0.446%	19.98%
Config 3b (O3, Tourn, 512k)	100k	0.8615	2,746M	0.446%	19.61%
Config 4 (O3, 8-way L1, 256k)	10k	0.8017	301M	0.795%	61.00%



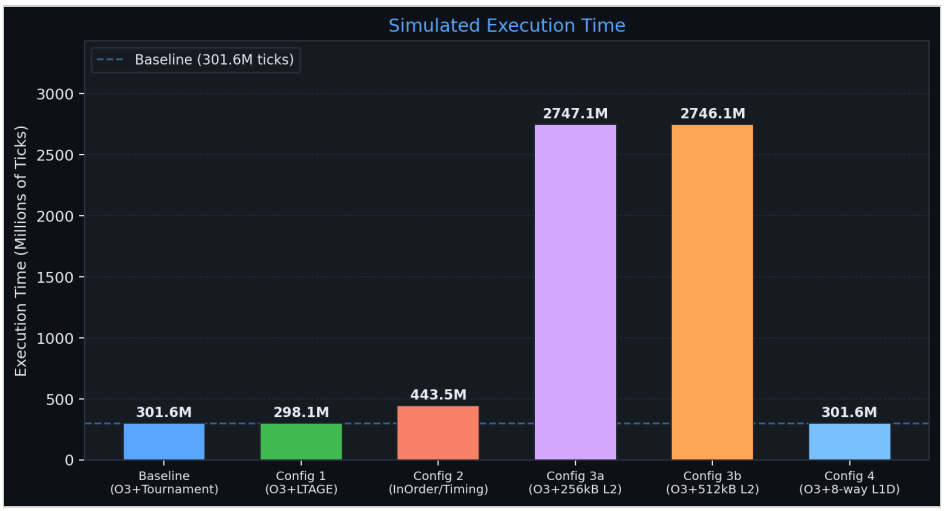
Graph 1: Baseline IPC against evaluated architectural modifications.



Graph 2: TournamentBP vs. LTAGE branch misprediction rates.



Graph 3: L2 (Primary) and L1D (Secondary) miss rate distributions.



Graph 4: Total simulated execution time (ticks).

5. Bottleneck Analysis

Config 1 (LTAGE): The predictor successfully halved the branch misprediction rate, verifying that Quicksort's conditional data dependencies heavily benefit from tagged geometric histories. However, IPC only improved by 1.17% overall.

Config 2 (In-Order): Performance dropped by 32% (IPC lowered from 0.80 to 0.54), establishing that memory latency hiding is the single biggest performance driver for this workload.

Config 3 (L2 Size Scaling): Doubling the L2 cache for a 100k element array provided fewer than 0.4% improvements to L2 misses. The structural random-access pivoting in Quicksort makes the workload completely pattern-limited rather than capacity-limited.

6. PPA Analysis & Conclusion

Pareto Conclusion: The optimal configuration is **O3CPU + LTAGE BP + 256kB L2**. The out-of-order core is strictly mandated to tolerate the high ~61% L2 miss penalties. The LTAGE predictor solves the secondary branch bottleneck at almost zero area cost (<0.1 mm^2). Conversely, scaling the LLC to 512kB constitutes a poor PPA investment due to its 2x static power overhead with negligible latency reduction. We conclude that algorithmic locality (e.g. Iterative Mergesort) or dedicated hardware stream prefetchers are necessary to completely bypass Quicksort's fundamental architectural ceilings.